
MORPH-32: Procedural Weight Quantization on Apple M5

MORPH-32 project

Technical report, September 2026

Abstract

MORPH-32 stores a short Boolean recipe for each tile of 32 model weights. Three seed words define sign planes; XOR and rotation generate two more. A tile scale maps their weighted sum to model values. Custom Metal kernels use this representation directly during text inference in MLX. On Qwen3.8-27B, the morph32-3s profile reduces the complete checkpoint from 16.08 to 13.67 GB and resident weights from 15.13 to 12.72 GB. A morph32-c profile lowers these to 13.14 and 12.19 GB. Perplexity rises by 2.42% and 5.32%, respectively, on a separate 4,096-token window. Generation falls from 18.05 tokens/s to 11.24 and 8.46 tokens/s. These are memory savings with a speed cost. The report specifies the representation, encoder, kernels, and scope of the saved experiments. It does not establish quality equivalence or a speed advantage over native MLX int4.

1 Introduction

A quantized model needs more memory than its weight codes alone. Scales, retained tensors, caches, and temporary arrays also occupy memory. A compressed file can even need a full decoded copy at inference time. A useful format must therefore define both the stored representation and the operations that consume it.

MORPH-32 asks whether a small shared state can represent a group of weights while remaining cheap enough to decode inside a GPU kernel. The stored state is a set of 32-bit words. Their bits specify sign planes across 32 adjacent weights. A fixed recurrence creates additional planes without storing them. The result is a constrained vector representation: changing one seed bit can affect several reconstructed weights.

The implementation targets MLX on Apple M5. It converts the language model’s multilayer perceptron (MLP) projections from bfloat16 (BF16), keeps other weights from the native int4 model, and writes a standalone checkpoint. Two profiles use the same decoder with different amounts of stored state. The experiments compare these profiles with the exact native MLX model that supplies the retained tensors.

The main finding is lower storage and measured memory at a speed cost. Priority over other procedural representations is not established.

2 Related work

Multiple binary bases were used before MORPH. ABC-Net approximates real-valued weights with a linear combination of binary bases (Lin et al., 2017). AQLM uses learned additive codebooks for groups of LLM weights (Egiazarian et al., 2024). These methods show why several reconstructed levels alone do not establish a new quantization principle.

GPTQ fits weights using approximate second-order information (Frantar et al., 2023). MORPH uses a cheaper diagonal objective based on activation energy. That objective weights important channels, but ignores correlations between them. It is not a replacement for measuring the actual model output.

The specific object studied here is a five-plane Boolean recurrence, its tile encoder, and direct Metal execution. The decoder has no learned codebook. Its generated planes are tied to its seeds. No matched comparison with GPTQ, AQLM, or independently stored binary planes was completed for this report. Those methods provide context, not benchmark baselines.

3 Representation

Let a projection have shape $N \times K$. Each output row is split into adjacent tiles of 32 weights along the input dimension. There is no channel permutation. Within a tile, bit j refers to weight j , starting at the least significant bit. All shifts, rotations, and XOR operations use unsigned 32-bit values.

morph32-3s tiles Store three words A , B , and C , plus one FP16 scale s . Define a rotate-and-XOR operator

$$R(z) = \text{rotl}(z, 5) \oplus \text{rotl}(z, 13) \oplus \text{rotl}(z, 21). \quad (1)$$

The two generated words are

$$D = B \oplus R(C), \quad E = C \oplus R(D). \quad (2)$$

For any word z , let $b_j(z)$ be its bit at position j , and define $u_j(z) = 2b_j(z) - 1$. The reconstructed weight is

$$\hat{w}_j = s \left(u_j(A) + \frac{1}{2}u_j(B) + \frac{1}{4}u_j(C) + \frac{1}{16}u_j(D) + \frac{1}{32}u_j(E) \right). \quad (3)$$

The last two coefficients are deliberately small. There is no coefficient of $\frac{1}{8}$ in the released profile. The representation needs $3 \times 32 + 16 = 112$ bits per tile, or 3.5 bits per weight including the scale.

The term orbit refers to repeated state updates. Only two updates are used. Generated planes add no independent stored information; they constrain the available vectors.

Table 1: Stored bits for one 32-weight tile. These rates include the FP16 scale. Complete model rates also include retained tensors and container overhead.

Tile layout	Bits	bpw
Three full seeds + scale	112	3.50
Two seeds + half seed + scale	96	3.00

morph32-c tiles and profile A partial tile keeps A , B , and only the low 16 bits L of C . The upper half of C is derived from the stored state:

$$H = (L \oplus A \oplus \text{rotl}(B, 5)) \text{ and } 65535, \quad (4)$$

$$C = L + 2^{16}H. \quad (5)$$

The tile then uses the same equations for D , E , and the weights. Storage is five unsigned 16-bit words followed by an FP16 scale: 96 bits, or 3.0 bpw. Only the independent bits are searched during fitting.

morph32-c uses partial tiles in the gate and up projections of zero-based layers 8 through 55. The other MLP projections use morph32-3s. This is 96 partial projections and 96 full-seed projections across 64 layers. Since these MLP matrices have equal parameter counts, their average rate is 3.25 bpw. morph32-3s uses 3.5 bpw for all 192 projections.

This selection is a fixed recipe from the experiments. It is not learned at load time. Complete model rates include retained weights and metadata.

4 Fitting the seeds

The target weights come from BF16. Calibration inputs come from the native MLX int4 model. For each input channel, the encoder uses its mean squared activation, normalized by the mean across channels, as an importance weight a_j . Gate and up projections share their input statistics.

For a tile, the fitting loss is

$$L(s, p) = \sum_{j=0}^{31} a_j (w_j - sv_j(p))^2, \quad (6)$$

where p denotes the independent seed bits and v_j is the unscaled five-plane sum. This diagonal loss is a proxy for activation-output error. It omits cross-channel terms in the input second-moment matrix. A low value therefore cannot establish low task error.

The encoder tries three initial scales. Each uses the largest absolute tile weight, multiplied by a clipping factor of 0.70, 0.85, or 1.00, and divided by the amplitude sum of the stored planes. Seed planes are initialized greedily from the remaining residual.

Each start then receives two sweeps over the stored seed bits. For each candidate flip, the recurrence is recomputed and the loss is summed across the whole tile. The flip is accepted only when this loss decreases. After each sweep, the scale is refitted:

$$s^* = \frac{\sum_j a_j w_j v_j}{\sum_j a_j v_j^2}. \quad (7)$$

The implementation guards the denominator, clamps the scale positive, and rounds it to FP16. The start with the lowest final loss is retained. Partial tiles search only 16 bits of the third seed.

Calibration uses 128 tokens from WikiText-2 validation at token offset 32,768. These tokens are separate from the reported test windows. Such a short sample keeps fitting cheap, but gives limited coverage of input behavior. The report does not establish that it covers coding, multilingual prompts, or long-context activations.

5 Execution and checkpoint

Single-token generation The generation kernel assigns one complete 32-weight tile to each SIMD lane. A 32-lane group thus covers 1,024 input weights per loop step. A lane loads its seeds and scale, generates the missing planes in registers, and accumulates a partial dot product. A SIMD reduction combines the partial sums.

The current mapping differs from assigning one weight to each lane. The format still uses 32-weight tiles and 32-bit words. The supported model dimensions are multiples of 1,024 for this fast path. The runtime never creates a dense copy of a complete MLP projection.

Prompt processing Prefill processes multiple tokens together. Groups of 128 threads reconstruct a 64×64 weight tile into threadgroup memory with padded stride 72. Four SIMD groups cooperate, and MLX’s M5 NAX helpers perform the matrix product. Reconstructed weights are rounded to the activation dtype before multiplication. The kernel handles partial token batches, with projection dimensions divisible by 64.

The prefill and generation paths expose different costs. Prefill reuses a decoded tile across tokens. Generation must reconstruct weights while producing each new token. Lower weight traffic can help memory use without overcoming decode and reduction work. The measured throughput includes the full model, so isolated kernel behavior cannot explain every part of the slowdown.

The kernels are integrated through MLX’s custom Metal interface (MLX contributors, 2026). The prefill path uses private M5 headers. MLX is pinned to version 0.32.2; portability to earlier Apple chips is not established.

Retained weights and metadata Attention, embeddings, and other unconverted tensors retain the native model’s values. Supported BF16 scale and bias metadata are compressed losslessly. Magnitude bit patterns are stored as offsets. Correlated scale and bias patterns use a prediction plus sparse residuals. Encoding is accepted only after exact reconstruction succeeds and the byte count falls.

Supported single-token projections fuse metadata reconstruction with multiplication. Other shapes reconstruct the small metadata arrays and call native MLX quantized multiplication. Their int4 weight codes remain packed. The measured memory reduction therefore includes both lossy MLP quantization and lossless metadata compression; the experiments do not separate their contributions in an end-to-end ablation.

Standalone container A .morph file is a safetensors container with a versioned JSON manifest. It includes tensor descriptors, original model configuration, tokenizer assets, and a SHA-256 hash for every tensor payload. Loading checks the inventory and shapes, verifies hashes, and assigns model tensors strictly. It needs neither source checkpoint at inference time.

Complete physical bits per weight is $8\frac{B}{P}$, where B is the entire file size and $P = 27,356,728,560$ is the logical parameter count. This includes retained vision tensors even though the evaluation uses text only. Resident model arrays and peak MLX allocation are reported separately. These metrics do not measure total process memory.

6 Experimental setup

The measured system was an Apple M5 Pro with 20 GPU cores and 48 GiB unified memory, running macOS 26.6.2. The software used MLX 0.32.2 and MLX-LM 0.31.3. The reference is the pinned MLX Community (2026) int4 snapshot of Qwen3.8-27B. The BF16 source is the pinned Qwen Team (2026) snapshot. The native int4 MLP format stores 4-bit codes with one BF16 scale and bias per 64 weights, for 4.5 bpw in those matrices.

Quality was measured on two 4,096-token WikiText-2 test windows (Merity et al., 2017), starting at token offsets 0 and 8,192. Each window starts with an empty cache. Next-token losses are accumulated in float32 in 128-token chunks through a continuous cache. Perplexity is the exponential of mean negative log-likelihood. Corpus scoring uses no chat template.

Throughput uses a 4,096-token prompt followed by 64 generated tokens. Kernels are warmed before three greedy generation runs. MLX is synchronized, and the table reports the median of each throughput field. Models were measured in separate sessions. The archive contains individual runs, but three repetitions do not characterize variation across machines or thermal states.

Additional checks use 50 fixed MMLU questions with five-shot choice likelihoods (Hendrycks et al., 2021), four reasoning questions, four Python functions, and one 17,644-token retrieval prompt. The generated workloads use non-thinking templates and greedy decoding. These checks cover specific failure cases; they are not a broad capability evaluation.

Results are archived measurements. Source revisions, data hashes, and measurement files accompany the Typst source.

7 Results

Storage and memory The complete file rates are higher than the MLP rates because attention, embeddings, vision weights, and other retained tensors still occupy space. Thus the 3.25-bpw MLP layout does not make the whole model 3.25 bpw. The file accounting includes scales, assets, and container bytes. Lossless compression of retained metadata also contributes to the savings; they cannot all be attributed to the orbit representation.

Relative to native, morph32-3s saves 2.41 GB of resident weights and morph32-c saves 2.95 GB. Peak MLX allocation falls by 13.1% and 16.0%, respectively (Table 2). The smaller peak reduction reflects allocations beyond model weights. These rows overlap and must not be added. They describe the recorded workload, not total memory for a persistent chat server or a longer context.

Table 2: Complete comparison on the measured Apple M5 Pro. GB is decimal; memory rows overlap. Bold marks the best observed value per row, not statistical significance. Throughput uses three warmed runs. The MMLU row describes only the fixed 50-question subset.

Measurement	Native MLX int4	morph32-3s	morph32-c
Complete checkpoint	16.08 GB	13.67 GB	13.14 GB
Complete physical bits/weight	4.703	3.998	3.841
MLP representation bits/weight	4.5	3.5	3.25 average
Resident weights	15.13 GB	12.72 GB	12.19 GB
Peak MLX memory	18.26 GB	15.87 GB	15.33 GB
Prompt processing, tokens/s	504.72	430.23	336.50
Generation, tokens/s	18.05	11.24	8.46
Perplexity, first 4,096 tokens	5.4638	5.6266	5.8075
Perplexity, separate 4,096-token window	4.7563	4.8712	5.0095
Separate-window perplexity increase	Reference	+2.42%	+5.32%
MMLU subset, 50 questions	41/50 (82.0%)	46/50 (92.0%)	42/50 (84.0%)

The additional compression cost Moving from morph32-3s to morph32-c saves another 0.535 GB in both checkpoint size and resident weights. That is about 4% of the larger profile. In exchange, prompt throughput falls by 21.8%, generation by 24.8%, and separate-window perplexity rises by 2.84%. The partial third seed removes free fitting bits from 96 projections. This is a useful option when that final memory saving matters, but a costly exchange when generation speed matters more.

Why smaller is not faster morph32-3s retains 85.2% of native prompt throughput and 62.3% of native generation throughput; morph32-c retains 66.7% and 46.9%. Direct decoding avoids a full dense MLP copy, but still executes recurrence, sign conversion, and scale operations. The partial layout adds seed reconstruction. Lower storage alone therefore does not imply lower execution time. The measurements show the net cost; without kernel-level profiling, they do not establish which instruction or memory access dominates it.

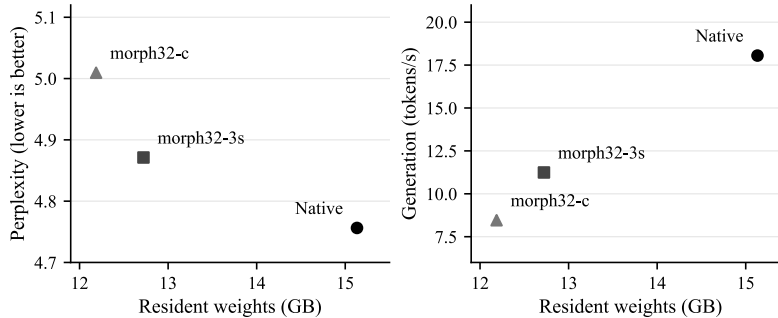


Figure 1: Measured perplexity and generation throughput against resident weight memory. These three observations do not estimate a frontier over other methods.

What the quality results mean Perplexity rises on both windows for both profiles. On the separate window, the increases correspond to 0.0239 and 0.0519 additional nats of mean negative log-likelihood per token. They measure how well the model predicts this text, not a percentage of general intelligence lost. Agreement between the two windows supports a local quality cost, but their limited text coverage cannot establish broader quality preservation.

The MMLU row goes in a different direction: morph32-3s answers five more questions correctly than native. Each question changes this score by two percentage points. A few changed answer rankings can therefore produce a large apparent gain while average language loss worsens. The subset spans 50 subjects and is too small to infer a general MMLU advantage. Bold type records the observed score only.

All profiles pass 413 assertions across four generated Python functions and recover one access code from a 17,644-token prompt. Reasoning scores are 2/4, 3/4, and 2/4. These checks catch specific

failures; four functions are not 413 independent coding tasks, and one retrieval success does not validate arbitrary long-context work.

Reproduction and integrity The locked environment, pinned sources, and frozen inputs define reproduction. Both checkpoints passed fresh-process loading and payload-hash verification. One checked question reproduced archived choice log probabilities exactly; cached single-token execution and resident byte counts were also checked.

A full morph32-3s conversion through the packaged CLI took 161.76 seconds. All 2,610 tensor payload hashes and all projection descriptors matched the measured checkpoint, with 129 additional header bytes. The full benchmark suite and morph32-c conversion were not repeated during packaging. An evidence audit recomputes the summary; the paper reads its snapshot without running models.

8 Discussion

The recurrence’s specific benefit remains untested. Independent sign planes, different rotations, and tail coefficients need matched ablations at equal physical rates. A better seed search could improve quality without changing decode work, allowing fitting and kernel costs to be studied separately.

Conclusion MORPH-32 reduces complete storage and measured memory on Qwen3.8-27B through procedural MLP weights and direct Metal execution. Both profiles have higher perplexity and slower generation than native int4. The unresolved question is whether a better fit or decoder can reduce those costs at the same complete model size.

Limitations

The evaluation covers one model and one Apple M5 Pro. It contains two short corpus windows and small task checks. No confidence interval, broad coding benchmark, multilingual evaluation, or 64K-context test is reported. Vision quality was not tested. There is no complete BF16 model evaluation, matched scalar 3-bit comparison, or controlled recurrence ablation. The results cannot establish quality equivalence, superiority over other quantizers, or support for M1 through M4. The report documents an implementation and its observed tradeoffs; it is not a peer-reviewed publication.

References

- Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. 2024. [Extreme compression of large language models via additive quantization](#). arXiv:2401.06118.
- Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. 2023. [GPTQ: Accurate post-training quantization for generative pre-trained transformers](#). ICLR.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. [Measuring massive multitask language understanding](#). ICLR.
- Xiaofan Lin, Cong Zhao, and Wei Pan. 2017. [Towards accurate binary convolutional neural network](#). NeurIPS.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. [Pointer sentinel mixture models](#). ICLR.
- MLX Community. 2026. [Qwen3.8-27B-4bit](#). Model snapshot 3e6447f082e8.
- MLX contributors. 2026. [Custom Metal kernels](#). MLX 0.32.2 documentation.
- Qwen Team. 2026. [Qwen3.8-27B](#). BF16 model snapshot 1d4bf0f2ff60.